

Medical GenAI RAG Platform

Local-First Retrieval-Augmented Generation Knowledge Layer for a Future Medical Agentic AI System

DOCUMENT TYPE	Solution Architecture Design (SAD)
SYSTEM	Medical RAG Knowledge Platform (Phase 1 of Medical Agentic AI Program)
VERSION	1.0 — Draft for Review
DATE	10 August 2026
DATA CLASSIFICATION	Synthetic / De-identified only — No PHI in this phase
DEPLOYMENT TARGET	Microsoft Azure (production); local Docker Compose for development

Table of Contents

- 1. Document Control**
- 2. Executive Summary**
- 3. Business Context & Objectives**
- 4. Scope**
- 5. Architecture Principles**
- 6. High-Level Solution Architecture**
- 7. Component Architecture**
 - 7.1 Ingestion Layer
 - 7.2 Embedding Layer
 - 7.3 Vector Storage Layer (Qdrant)
 - 7.4 Retrieval & Reranking Layer
 - 7.5 Generation Layer (Local Ollama)
 - 7.6 Grounding & Guardrails Layer
 - 7.7 API Layer (FastAPI)
 - 7.8 UI Layer (Streamlit)
- 8. Data Architecture**
- 9. Technology Stack**
- 10. Deployment Architecture**
 - 10.1 Local Development Topology
 - 10.2 Azure Production Deployment
 - 10.3 Portability Beyond Azure
- 11. API Design**
- 12. Security & Compliance Considerations**
- 13. Observability Architecture**
- 14. Testing & Evaluation Strategy**
- 15. Non-Functional Requirements**
- 16. Future State — Medical Agentic AI Integration**
- 17. Risks, Assumptions & Open Items**
- 18. Appendix**

1. Document Control

Field	Value
Document Title	Medical GenAI RAG Platform — Solution Architecture Design Document
Version	1.0 — Draft for Review
Date	10 August 2026
Owner	Generative AI Architecture Team
Status	Phase 1 (Environment Setup) delivered; Phases 2–14 in progress
Data Classification	Synthetic / de-identified data only. No real PHI used or referenced.
Related Documents	Implementation README (phased build log), .env.example, docker-compose.yml

1.1 Revision History

Version	Date	Description	Author
0.1	10 Aug 2026	Initial draft covering target architecture and Phase 1 scope	GenAI Architecture Team
1.0	10 Aug 2026	Reviewed and issued as baseline solution architecture	GenAI Architecture Team

2. Executive Summary

This document defines the target solution architecture for a local-first, production-oriented Retrieval-Augmented Generation (RAG) platform for medical and clinical-workflow knowledge. The platform is the first deliverable in a two-stage program: it establishes a standalone, independently deployable and independently testable **Knowledge Layer**, which will later be consumed as a tool by a separate **Medical Agentic AI system** operating across roughly 24 connected medical application screens.

The defining architectural constraint is that the platform's AI components must run entirely on self-hosted, local-first infrastructure: **Qdrant** as the vector database, **Ollama** as the local LLM runtime, and a local **Sentence Transformers** model for embeddings. No document content, query text, or embeddings are sent to any external LLM or vector database API at any point in the pipeline. Production hosting for this self-hosted stack is **Microsoft Azure** — Qdrant and Ollama run on Azure compute (VMs, or AKS at scale) that the platform controls directly, not on managed AI services such as Azure OpenAI or Azure AI Search (see Section 10.2).

The system is being delivered incrementally across 14 build phases, each independently explained, implemented, and verified before the next begins (see Section 17 and the companion implementation README for the live phase-by-phase log). This document describes the **target end-state architecture** that all 14 phases converge on, not a single phase snapshot.

2.1 Key Architectural Decisions

- Vector store:** Qdrant, self-hosted on Docker Compose locally and on an Azure VM in production, using a single `medical_knowledge` collection with rich payload metadata for filtering.
- LLM runtime:** Ollama, addressed via a dedicated `OllamaLLMService` abstraction; the model name is environment-configured, never hard-coded, so the platform is model-agnostic across Llama, Qwen, Mistral, or any locally installed model. Runs on an Azure GPU VM in production.

- **Embeddings:** a single local Sentence Transformers model shared identically between indexing and query time, exposed via an `EmbeddingService` abstraction to eliminate embedding-space drift.
- **Retrieval:** semantic vector search plus structured metadata filtering, with workflow/screen-aware prioritization and a cross-encoder reranking stage before any chunk reaches the LLM.
- **Trust layer:** explicit grounding scoring, conflict detection across sources, and prompt-injection screening of retrieved content, with a mandated "insufficient evidence" fallback instead of a fabricated answer.
- **Cloud hosting:** Microsoft Azure is the target production environment (Section 10.2) — a segmented Virtual Network, GPU compute for Ollama, disk-backed Qdrant, and platform services (Key Vault, Monitor, ACR) around them. Azure hosts the same self-hosted stack; it does not replace any component with a managed AI service.
- **Separation of concerns:** the RAG platform provides knowledge only. Planning, decision-making, tool orchestration (OCR, vision, click/type actions) and memory belong to the future Agent layer, explicitly out of scope here.

3. Business Context & Objectives

The long-term goal of the program is a Medical AI Agent capable of operating across a multi-screen clinical application: understanding the current screen and workflow state, reading on-screen information, retrieving relevant medical and procedural knowledge, deciding on the correct next action, invoking tools (OCR, vision, click, type, zoom, segment, check, save), verifying results, maintaining memory across the interaction, and safely progressing the workflow to completion.

Building the Agent and the knowledge layer together would couple two very different engineering concerns — non-deterministic planning/action-taking versus deterministic, testable retrieval — into one hard-to-verify system. This program therefore deliberately sequences the work: build and harden the RAG knowledge platform first, as a standalone service with its own API, its own test suite, and its own evaluation harness; introduce the Agent only once that knowledge layer is proven.

3.1 Objectives of this Phase

- Deliver a RAG platform that can ingest heterogeneous medical/workflow documents and answer grounded, cited questions against them.
- Support workflow- and screen-aware retrieval so that, in the future, an Agent operating on Screen 7 of Workflow WF_001 receives knowledge prioritized for that exact context.
- Guarantee the platform can be deployed, operated, and evaluated with zero dependency on external AI vendors.
- Expose a stable, well-typed API surface (`rag.query`, `rag.search`, and related operations) that the future Agent can call as a tool without knowledge of RAG internals.

4. Scope

4.1 In Scope

- Document ingestion for PDF, DOCX, TXT, Markdown, and JSON sources.
- Cleaning, normalization, metadata extraction, and configurable chunking.
- Local embedding generation and storage/retrieval in Qdrant, including metadata filtering.
- Semantic retrieval, cross-encoder reranking, and screen/workflow-aware ranking.
- Grounded answer generation via local Ollama, with structured, cited output.
- Grounding assessment, conflict detection, and prompt-injection screening.

- A FastAPI service exposing ingestion, indexing, search, query, and health endpoints.
- A Streamlit demonstration UI for upload, indexing, and question answering.
- Observability (structured logs, latency, retrieval/generation metrics) and a RAG evaluation harness.
- Synthetic/de-identified demo knowledge base and sample question set.

4.2 Out of Scope (this phase)

- The Medical Agentic AI system itself — planning, decision-making, tool orchestration, and cross-screen navigation.
- OCR, computer vision, and UI-automation tooling (click/type/zoom/segment).
- Persistent Agent memory / long-running session state.
- Any ingestion, storage, or processing of real patient data (PHI). The prototype uses only synthetic or de-identified content.
- Multi-tenant access control and production identity/authentication integration (the API is built "authentication-ready" but auth itself is not implemented in this phase).

5. Architecture Principles

Principle	Description
Local-first, vendor-neutral	Every AI capability — embeddings, generation — runs on local infrastructure. No document content or query ever leaves the deployment boundary via a cloud LLM or embedding API.
Single source of configuration	All environment-specific values (hosts, ports, model names, chunk sizes, thresholds) live in one centralized settings module, never hard-coded in application logic.
Separation of knowledge from action	The RAG platform answers "what do we know"; it never decides "what to do next". That responsibility belongs entirely to the future Agent layer.
Grounded by default, confident only when justified	The system never silently guesses. Low-confidence or conflicting evidence produces an explicit, honest response rather than a fabricated one.
Retrieved content is data, not instructions	Documents in the knowledge base — including any adversarial or malformed content — are treated strictly as data. The LLM must never follow instructions embedded inside retrieved text.
Independently deployable, independently testable	Every layer (ingestion, embeddings, vector store, retrieval, generation, API) can be built, run, and tested in isolation, without requiring the full stack to be online.
Model-agnostic by configuration	Swapping the Ollama model or the embedding model is an environment-variable change, not a code change.

6. High-Level Solution Architecture

The platform implements a linear ingestion pipeline feeding a vector store, and a linear query pipeline that retrieves, reranks, grounds, and cites an answer generated by a local LLM. Both pipelines are exposed through one FastAPI service, which is the sole integration surface for the future Medical Agent.

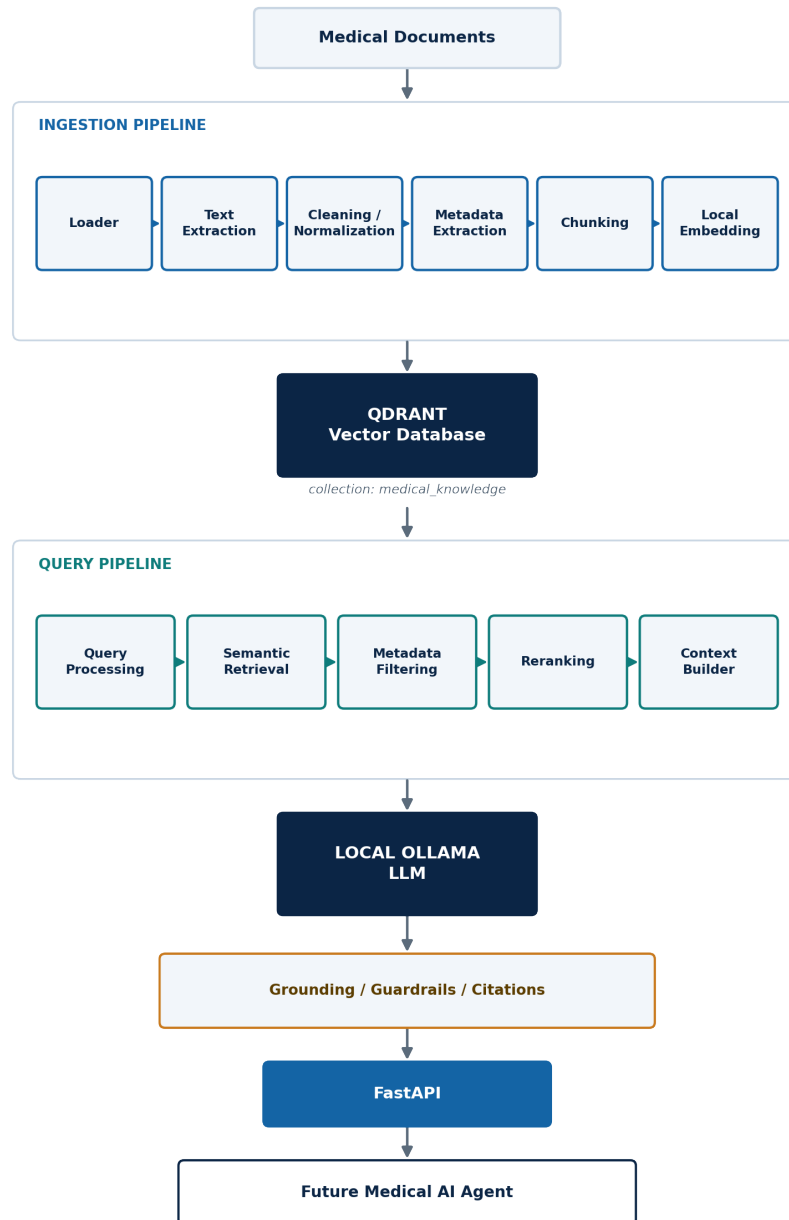


Figure 1 — End-to-end ingestion and query pipelines, both terminating in the FastAPI service consumed by the future Agent.

6.1 Layered View

- **Presentation:** Streamlit UI (human demo/testing) and FastAPI (machine consumer — the future Agent).
- **Orchestration:** the RAG Pipeline, coordinating retrieval, reranking, context construction, generation, and grounding.
- **Reasoning:** the local Ollama LLM, used strictly for grounded generation over supplied context.
- **Knowledge:** Qdrant vector store plus its payload metadata schema.
- **Ingestion:** the offline/batch pipeline that turns raw documents into indexed, searchable knowledge.

- **Cross-cutting:** configuration, guardrails (prompt-injection screening, safety validation), and observability, all touching every layer above.

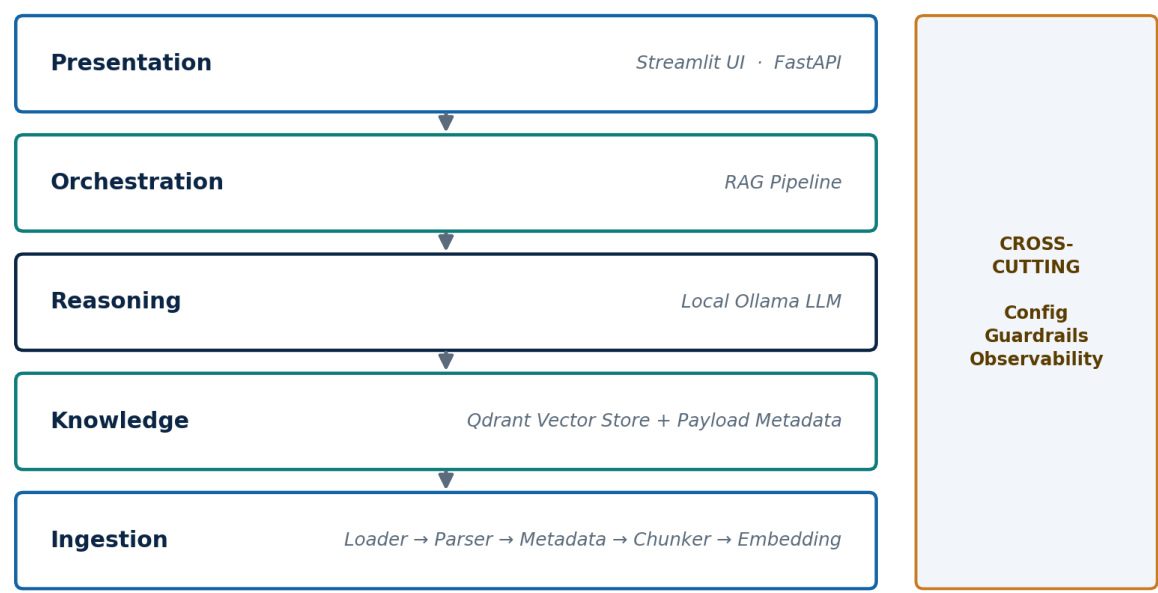


Figure 5 — Layered view: five functional layers plus the cross-cutting concerns that touch all of them.

7. Component Architecture

7.1 Ingestion Layer

Converts heterogeneous source documents into clean, chunked, metadata-tagged text ready for embedding. Implemented as four composable stages so each can be tested and replaced independently.

Module	Responsibility	Key Libraries
loaders.py	Format-specific extraction for PDF, DOCX, TXT, Markdown, JSON	PyMuPDF, python-docx
parser.py	Whitespace/encoding normalization, boilerplate stripping, structure preservation	—
metadata.py	Builds the standard metadata payload per chunk (document_id, workflow_id, screen_id, etc.)	pydantic
chunker.py	Splits cleaned text into overlapping chunks of configurable size	—

Chunking Strategy

The initial implementation uses fixed-size chunking with configurable `CHUNK_SIZE` (default 800 characters/tokens) and `CHUNK_OVERLAP` (default 100), with boundaries adjusted to respect sentence and paragraph breaks where possible. This is intentionally simple, predictable, and fast to evaluate.

Planned evolution: semantic chunking (splitting on embedding-based topic shifts rather than a fixed character count) and parent-child chunking (indexing small, precise chunks for retrieval while returning their larger parent chunk for LLM context) are architected for but not required in the initial release. Both can be introduced behind the same chunker interface without changing downstream components.

7.2 Embedding Layer

A single `EmbeddingService`, backed by a local Sentence Transformers model, exposes `embed_text()`, `embed_documents()`, and `embed_query()`. The same model instance and model identity is used for both indexing and querying — this is enforced architecturally (one shared service, one configured model name) because a mismatch here silently degrades retrieval quality without raising any error.

Setting	Default	Notes
EMBEDDING_MODEL	BAAI/bge-small-en-v1.5	General-purpose, strong retrieval performance at small size; swappable via env var
Vector dimension	384	Determined by the selected model; the Qdrant collection is created to match
EMBEDDING_DEVICE	cpu	Set to a CUDA device string to use GPU acceleration where available
EMBEDDING_BATCH_SIZE	32	Controls throughput vs. memory use during bulk indexing

Model selection note

BAAI/bge-small-en-v1.5 is the default because it balances retrieval quality against CPU-only inference speed for local deployment. Domain-specific alternatives (e.g. a PubMedBERT-family sentence embedding model) can be substituted via `EMBEDDING_MODEL`; doing so requires re-indexing the full corpus, since vectors from different models are not comparable.

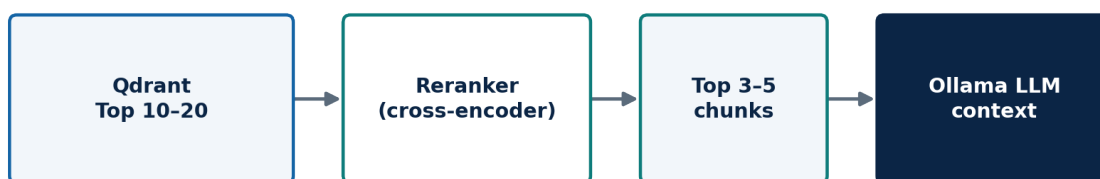
7.3 Vector Storage Layer (Qdrant)

Qdrant is deployed locally via Docker Compose and accessed through a QdrantStore wrapper around qdrant-client. A single collection, `medical_knowledge`, holds all vectors; document scoping (by workflow, screen, domain, access level) is achieved through payload metadata filtering rather than separate collections, keeping operational overhead low while the corpus is at prototype scale.

- Collection creation and schema management
- Health checks (collection existence, point count, index status)
- Vector upsert (single and batch)
- Similarity search with score threshold and top-K limits
- Metadata (payload) filtering, combinable with vector search in a single call
- Collection statistics for observability and evaluation reporting

7.4 Retrieval & Reranking Layer

Retrieval is a two-stage funnel: an initial broad semantic search against Qdrant (top 10–20, score-thresholded and metadata-filtered), followed by cross-encoder reranking down to a small, high-precision set (top 3–5) before anything is sent to the LLM. This keeps prompt context small, relevant, and cheap to reason over, and keeps a clean separation between "could be relevant" (vector search recall) and "is actually relevant to this exact question" (reranker precision).



Score threshold + metadata filter applied before reranking; only high-precision chunks reach the LLM

Figure 6 — The retrieval-to-generation funnel: broad recall narrows to a small, high-precision context.

Screen-Aware Prioritization

When a query supplies `workflow_id` / `screen_id`, matching documents are boosted in ranking rather than used as a hard filter — this preserves access to broader medical knowledge (e.g. general terminology or cross-cutting safety rules) while still prioritizing the exact operational context the caller is in.

7.5 Generation Layer (Local Ollama)

All text generation is performed by a locally running Ollama instance, accessed exclusively through `OllamaLLMService`. The service exposes `generate()`, `generate_structured()` (for models supporting structured/JSON output), and `health_check()`. The model identity (`OLLAMA_MODEL`) is never hard-coded; the platform has been validated to work with Llama, Qwen, and Mistral family models available through Ollama.

Grounded Generation Prompting

Generation uses a fixed system prompt template that instructs the model to treat retrieved context as the sole primary source, to never invent medical or workflow information, to explicitly flag insufficient evidence, to surface (not silently resolve) conflicting documents, to prefer the latest approved document version, and to separate retrieved evidence from any inference. This prompt contract is the same across all supported models.

7.6 Grounding & Guardrails Layer

Two complementary trust mechanisms sit between generation and the API response:

- **Grounding assessment** — evaluates whether the generated answer is actually supported by the retrieved context, producing a boolean grounded flag, a numeric confidence score, and a HIGH / MEDIUM / LOW bucket. Below the configured confidence threshold, the system returns an explicit insufficient-evidence response instead of a low-confidence answer.
- **Conflict detection** — when retrieved documents disagree (e.g. two versions of a validation rule for the same screen), the system does not silently pick one; it surfaces the conflict and both sources to the caller.
- **Prompt-injection screening** — retrieved chunks are scanned for embedded instructions (e.g. attempts to override the system prompt) before being placed into context; retrieved content is always treated as data, never as executable instruction.

7.7 API Layer (FastAPI)

A single FastAPI service is the platform's external contract — the same contract the Streamlit UI and, later, the Medical Agent both consume. Endpoint design is detailed in Section 11.

7.8 UI Layer (Streamlit)

A lightweight Streamlit application provides a human-facing demonstration and manual test surface: document upload and indexing, question submission, and inspection of retrieved chunks, the generated answer, citations, confidence, and grounding status side by side. It calls the FastAPI service — it has no direct access to Qdrant or Ollama — so it exercises the exact same path an external caller (including the future Agent) would use.

8. Data Architecture

8.1 Qdrant Payload Schema

Every vector stored in the `medical_knowledge` collection carries the following payload metadata, enabling structured filtering alongside semantic search:

Field	Type	Purpose
chunk_id	string	Unique identifier for this chunk; used in citations
document_id	string	Identifier of the source document
document_name	string	Human-readable document name
chunk_text	string	The chunk content returned as retrieved evidence
page_number	integer	Source page, for citation and traceability
document_type	string	e.g. guideline, workflow, SOP, terminology, tool-instruction
workflow_id	string	Associates the chunk with a specific clinical workflow
screen_id	string	Associates the chunk with a specific application screen
medical_domain	string	e.g. radiology, cardiology — supports domain filtering
document_version	string	Enables "prefer latest approved version" resolution
effective_date	date	Supports recency-aware ranking and conflict resolution
source	string	Provenance, e.g. internal / external / vendor documentation
access_level	string	Access-control-ready classification (e.g. standard / restricted)

8.2 Source Document Metadata (Ingest Time)

Each document supplied to the ingestion pipeline carries a metadata envelope, for example:

```
{
  "document_id": "DOC_001",
  "document_name": "Screen_07_Workflow",
  "document_type": "workflow",
  "version": "1.0",
  "workflow_id": "WF_001",
  "screen_id": "SCREEN_07",
  "medical_domain": "radiology",
  "effective_date": "2026-01-01",
  "source": "internal",
  "access_level": "standard"
}
```

8.3 Knowledge Base Composition (Prototype)

The demonstration knowledge base is entirely synthetic/de-identified and spans:

- Medical guidelines and medical terminology references
- Clinical workflow documents and screen definitions
- Screen-specific instructions and validation rules
- Hospital SOP examples and medical application documentation
- Tool instructions and agent operating procedures

- Synthetic worked examples used for evaluation

8.4 Data Lifecycle

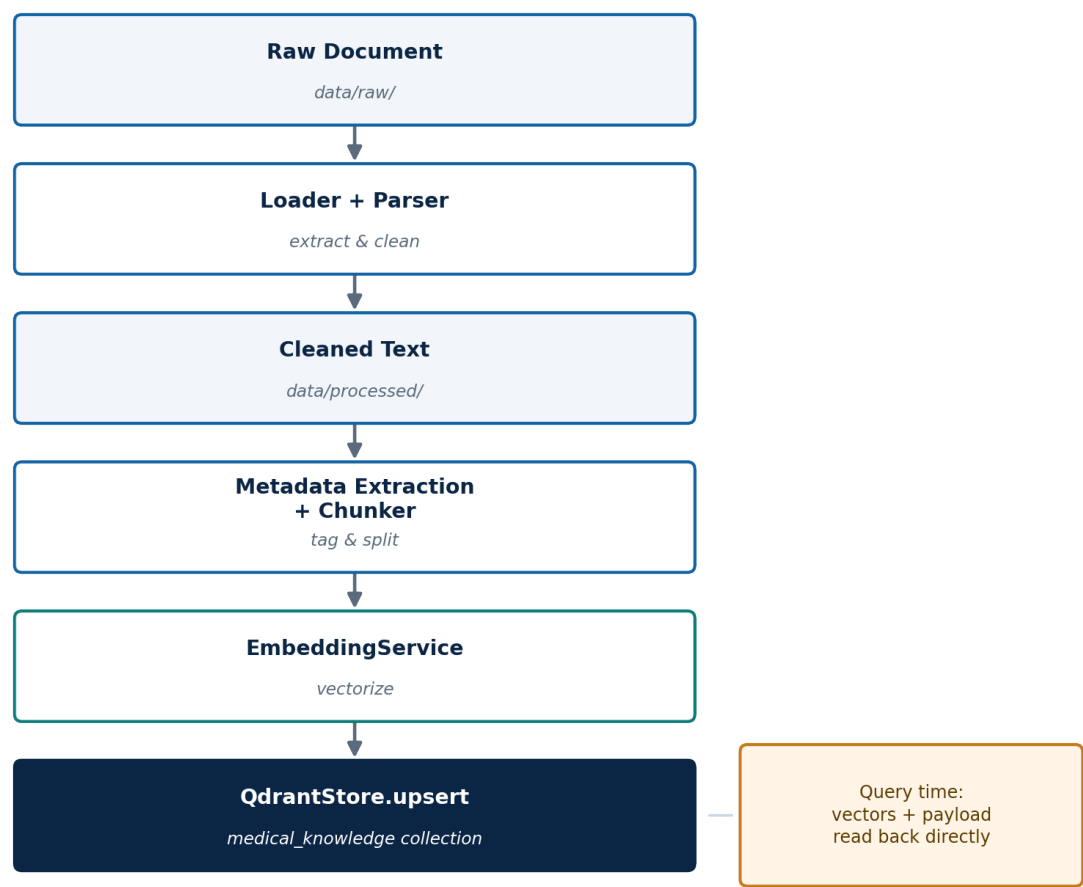


Figure 2 — Data moves one direction through ingestion; queries never touch raw source files directly.

9. Technology Stack

Layer	Technology	Rationale
Language / Runtime	Python 3.11+	Mature AI/ML ecosystem; typing improvements used throughout
API Framework	FastAPI	Async-native, automatic OpenAPI schema — a natural tool contract for the future Agent
Vector Database	Qdrant (self-hosted — Docker locally, Azure VM in production)	Mandated; strong payload-filtering support, simple to self-host anywhere
LLM Runtime	Ollama (self-hosted — host process locally, Azure GPU VM in production)	Mandated; runs open models fully under platform control with a uniform local API
Embeddings	Sentence Transformers (local)	Mandated; avoids any external embedding API dependency
Document Parsing	PyMuPDF, python-docx	Reliable, widely used open-source extraction libraries
Reranking	Cross-encoder (Sentence Transformers)	Local, no additional service dependency
Config Management	pydantic-settings	Typed, validated, single source of environment configuration
UI	Streamlit	Fast to build, sufficient for a demonstration/test harness
Containerization	Docker / Docker Compose	Reproducible local infrastructure; same images promoted to Azure
Cloud Hosting (production)	Microsoft Azure — VNet, VMs / AKS, Managed Disk	Self-hosts the mandated stack with production networking, identity, and ops tooling (Section 10.2)
Secrets Management	Azure Key Vault (prod); .env (local)	Centralized, managed-identity-backed secret resolution in production
CI/CD	GitHub Actions or Azure DevOps → Azure Container Registry	Standard build/push/deploy pipeline for the app images (Section 10.2.3)
Testing	pytest	Standard Python testing; supports the module-by-module verification strategy
Logging	loguru (structured) → Azure Monitor / Log Analytics	Same structured schema locally and in production; only the sink changes

Explicitly excluded

Azure OpenAI, Azure AI Search, OpenAI API, any other cloud LLM API, Pinecone, and any vector database other than self-hosted Qdrant are excluded by mandate. Azure is used as infrastructure to host the mandated open-source stack — never as a substitute AI service.

10. Deployment Architecture

The platform is developed against a local topology and deployed to production on **Microsoft Azure**. Moving to Azure changes where the mandated components run — it does not change what they are. Qdrant and Ollama remain self-hosted, open-source components under this architecture's control; they are deployed onto Azure compute (VMs, or AKS at scale), not replaced by managed AI services such as Azure OpenAI or Azure AI Search. This preserves the local-first / no-external-LLM-API principle (Section 5) while giving the platform production-grade hosting, identity, networking, and observability.

10.1 Local Development Topology

In local development, the FastAPI application and Streamlit UI run directly on the host, while Qdrant runs in Docker via the provided Compose file. Ollama runs on the host as a native service. This remains the inner-loop developer experience regardless of where the platform is ultimately deployed.

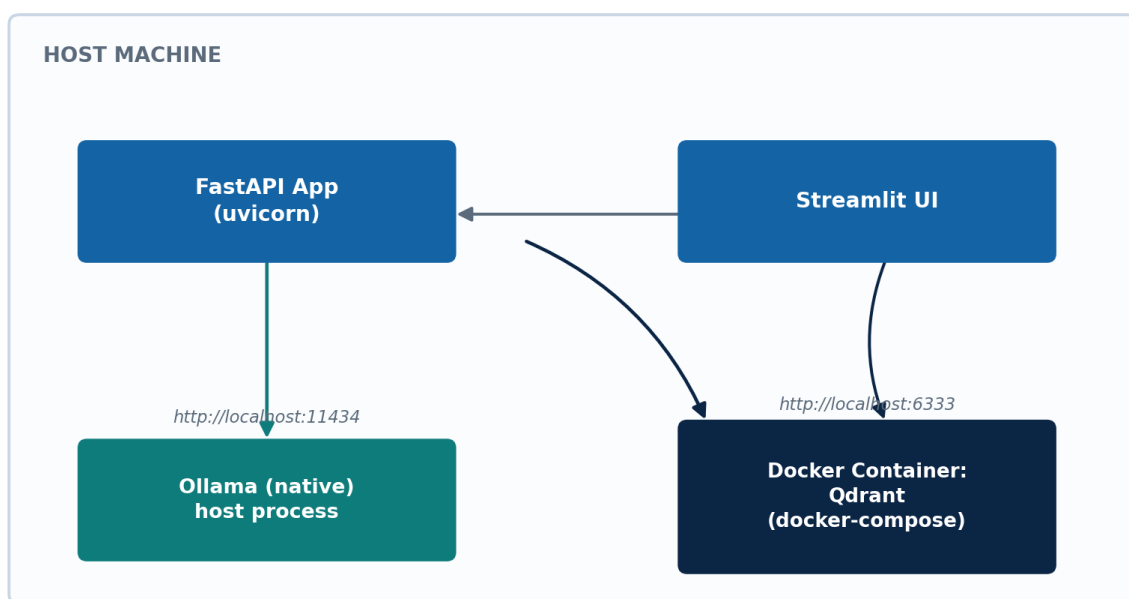


Figure 3 — Local topology: only Qdrant is containerized; the app and Ollama run on the host.

10.2 Azure Production Deployment (Target Environment)

Production deploys the same components into a dedicated Azure Resource Group, inside a Virtual Network segmented into an application subnet and an AI-compute subnet. Ollama runs on a GPU-enabled VM (e.g. the NCasT4_v3 series) for acceptable local-inference latency; Qdrant runs on its own VM backed by an Azure Managed Disk (Premium SSD) for durable, low-latency vector storage. FastAPI and Streamlit sit in the application subnet and are the only components reachable from outside the network, via Application Gateway / API Management.

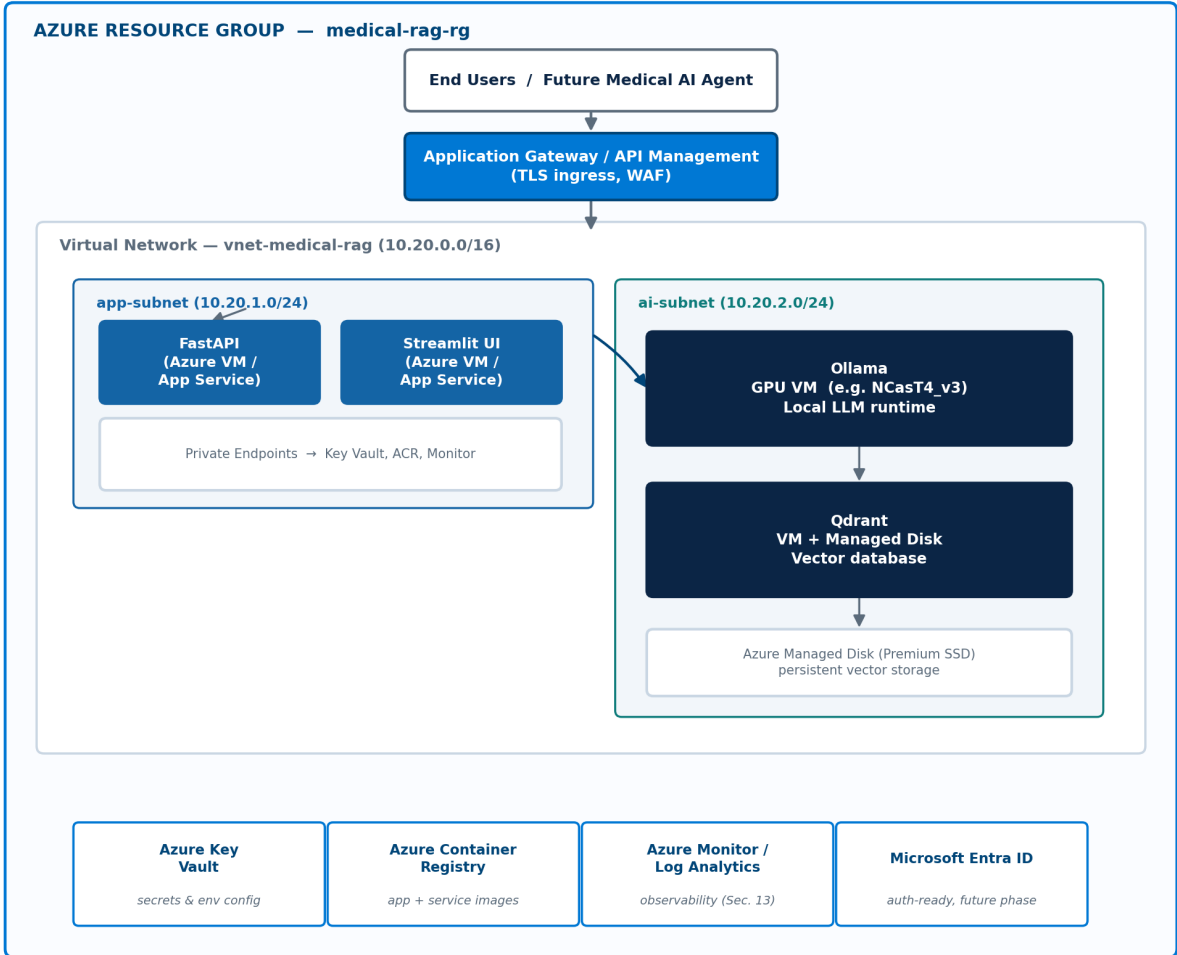


Figure 7 — Target Azure production architecture: segmented VNet, GPU-backed Ollama, disk-backed Qdrant, and supporting platform services.

Core Azure Resources

Resource	Purpose	Notes
Resource Group	Logical container for all platform resources	e.g. medical-rag-rg; one per environment (dev/staging/prod)
Virtual Network (VNet)	Network isolation with app-subnet and ai-subnet	No component is publicly reachable except via the gateway
Ollama GPU VM	Hosts the local LLM runtime	GPU SKU sized to the chosen model (Section 18.4 sizing guidance)
Qdrant VM + Managed Disk	Hosts the vector database with persistent storage	Premium SSD for IOPS; snapshotted for backup
FastAPI compute	Hosts the API layer	Azure VM (parity with local Docker Compose) or Azure App Service for simpler ops
Streamlit compute	Hosts the demo/test UI	Same compute choice as FastAPI for consistency
Application Gateway / API Management	Single TLS ingress point, WAF, and (later) rate limiting	The only internet-facing resource; also the future Agent's entry point
Azure Key Vault	Centralized secrets (replaces plain .env in production)	Referenced via managed identity, never checked into source control
Azure Container Registry (ACR)	Stores built application images	Populated by CI/CD (Section 10.2.3)
Azure Monitor / Log Analytics	Central destination for the structured logs in Section 13	Same log schema as local dev — only the sink changes
Microsoft Entra ID	Identity provider for future authentication	Not enabled in this phase; endpoints are built authentication-ready (Section 12)

10.2.1 Network & Security Design

- **Segmentation:** app-subnet (FastAPI, Streamlit) is separated from ai-subnet (Ollama, Qdrant); only the app-subnet is reachable from the Application Gateway.
- **No public IPs on compute:** Ollama and Qdrant VMs have no public IP; all traffic to them originates from the app-subnet.
- **Private Endpoints:** Key Vault, Container Registry, and Monitor are reached over Azure Private Link from the app-subnet rather than the public internet.
- **Network Security Groups (NSGs):** explicit allow-rules between app-subnet and ai-subnet on only the required ports (Qdrant 6333/6334, Ollama 11434); default-deny otherwise.
- **Secrets:** all values currently in .env for local development (Section 18.1) move to Key Vault in production, injected via managed identity at process startup — no secret ever lives in an image or App Setting in plain text.
- **Disk encryption:** Azure Managed Disks are encrypted at rest by default (platform-managed keys), upgradable to customer-managed keys in Key Vault if required by policy.

10.2.2 Configuration Mapping (Local → Azure)

The same centralized settings module (Section 7, `app/config/settings.py`) is used unchanged; only the values it resolves at runtime differ by environment:

Setting	Local development	Azure production
QDRANT_HOST	localhost	Private DNS name of the Qdrant VM inside ai-subnet (e.g. qdrant.internal.medical-rag.local)
OLLAMA_BASE_URL	http://localhost:11434	Private DNS/IP of the Ollama GPU VM inside ai-subnet, over the internal network only
Secrets (API keys, if any future auth is added)	.env file, git-ignored	Azure Key Vault reference, resolved via managed identity
Log destination	Local stdout / file (loguru)	Azure Monitor / Log Analytics workspace, same structured schema

10.2.3 CI/CD Pipeline

Application images (FastAPI, Streamlit) are built and pushed to Azure Container Registry on merge to the main branch, then deployed to the target VMs (or an AKS cluster, Section 10.2.5) using a standard build → push → deploy pipeline (GitHub Actions or Azure DevOps, either is compatible with this architecture). The Qdrant and Ollama images are the stock upstream images and are not rebuilt by this pipeline — only configuration and networking around them are managed as infrastructure-as-code (e.g. Bicep or Terraform).

10.2.4 Resource Sizing Guidance (Starting Point)

Component	Suggested starting SKU	Rationale
Ollama GPU VM	Standard_NC4as_T4_v3 (1x T4 GPU)	Sufficient for 7B-class models at prototype/demo load; scale up (more VRAM) for larger models or higher concurrency
Qdrant VM	Standard_D4s_v5 + Premium SSD (P15, 256 GiB)	CPU-oriented; disk IOPS matter more than raw compute at prototype corpus size
FastAPI / Streamlit compute	Standard_B2s–B4ms, or App Service P1v3	Light workloads; the LLM and vector search do the heavy lifting
Application Gateway	WAF_v2, Standard tier	TLS termination + basic WAF rules for a single ingress point

Sizing is a starting point, not a commitment

These SKUs are reasonable defaults for the prototype/demo scale described in this document (Section 4). Actual sizing should be confirmed against the RAG evaluation harness (Section 14) latency results and real corpus size before any production commitment, and revisited if concurrent Agent usage across the full 24-screen workflow is introduced.

10.2.5 Scaling Path: AKS (Future Option)

For higher availability or multi-instance scaling, FastAPI and Streamlit can move from standalone VMs to Azure Kubernetes Service (AKS) without any application code changes, since both are already containerized for local Docker Compose use. Qdrant also has a supported Kubernetes deployment mode for clustered, sharded operation if the corpus grows well beyond prototype scale. Ollama is typically kept on a dedicated GPU VM (or GPU node pool) rather than autoscaled, since LLM inference benefits from a warm, dedicated model in memory rather than cold-starting per request.

10.2.6 Cost Considerations

The dominant cost driver is the GPU VM hosting Ollama, which — unlike a pay-per-token cloud LLM API — is billed for uptime regardless of query volume. This is a deliberate trade-off consistent with the no-external-LLM-API mandate: predictable infrastructure cost and full data control, in exchange for paying for idle capacity between queries. Where usage is bursty, the GPU VM can be started/deallocated on a schedule, or the workload can be moved to an AKS GPU node pool with autoscaling once usage patterns are understood.

10.3 Portability Beyond Azure

Because every network endpoint and secret is resolved through the single centralized settings module (Section 7.1's config principle), the same codebase is not locked to Azure. The Azure resource map in Section 10.2 could be replaced by an equivalent on another cloud or on-premises data center by changing infrastructure-as-code and environment values only — no application code changes. Azure is this program's chosen target for production, not an architectural dependency.

11. API Design

The FastAPI service is the single integration surface for the Streamlit UI today and the Medical Agent in the future. All side-effecting operations (ingest, index) are separate from read operations (search, query), so the future Agent can safely call read endpoints frequently without risk of mutating the knowledge base.

Method & Path	Purpose
GET /health	Overall service health
GET /qdrant/health	Qdrant connectivity and collection status
GET /ollama/health	Ollama connectivity and model availability
POST /documents/ingest	Upload, parse, and clean a source document
POST /documents/index	Chunk, embed, and write a processed document to Qdrant
GET /documents/{document_id}	Fetch stored document metadata and chunk listing
POST /rag/search	Retrieval only — returns ranked chunks, no LLM generation
POST /rag/query	Full pipeline: retrieve, rerank, generate, ground, cite

11.1 Primary Contract — POST /rag/query

Request

```
{
  "query": "What should I verify before completing Screen 7?",
  "workflow_id": "WF_001",
  "screen_id": "SCREEN_07"
}
```

Response

```
{
  "query": "...",
  "answer": "...",
  "grounded": true,
  "confidence": 0.91,
  "sources": [
    {
```

```
    "document_id": "DOC_001",
    "document_name": "Screen_07_Workflow",
    "page": 12,
    "chunk_id": "CHUNK_034",
    "score": 0.91
  }
],
"retrieved_chunks": [],
"warnings": []
}
```

11.2 Screen-Aware Extension (Future Agent Calls)

The request contract is designed to extend naturally to full Agent context without breaking existing callers — additional fields are optional:

```
{
  "query": "...",
  "workflow_id": "...",
  "screen_id": "...",
  "current_state": {},
  "previous_state": {},
  "detected_text": []
}
```

12. Security & Compliance Considerations

12.1 Prototype Posture

- Only synthetic or de-identified data is used or referenced — no PHI enters the system at any stage.
- All secrets and endpoints are environment-based (.env), never hard-coded or committed to source control.
- The API surface is built authentication-ready (clean request/response boundaries) though authentication itself is not enabled in this phase.
- Metadata includes an access_level field so access-control policy can be layered on without a schema change.
- The architecture is audit-ready: every retrieval and generation step is logged (Section 13) with enough structure to reconstruct how an answer was produced.
- Retrieved content is always treated as untrusted data with respect to the LLM's instruction-following (Section 7.6), reducing prompt-injection risk from malicious or corrupted documents.

12.2 Additional Controls Required Before Real PHI Use

Not production-ready for PHI as-is

This platform is a prototype architecture validated with synthetic data only. Using it with real patient data would require, at minimum, all of the controls below, implemented and independently verified before go-live.

- **Authentication & authorization:** real identity integration (OAuth2/OIDC or equivalent), enforced role- and access-level-based authorization on every endpoint, not just descriptive metadata.
- **Encryption:** TLS in transit for all service-to-service traffic (API, Qdrant, Ollama), and encryption at rest for the Qdrant storage volume and any cached documents.
- **PHI-specific handling:** a formal data classification and minimum-necessary review of every field ingested; de-identification or tokenization pipelines where full PHI is not required.
- **Audit logging:** immutable, centralized audit logs of every query, document access, and administrative action, retained per applicable regulatory schedules.
- **Regulatory alignment:** a formal HIPAA (or applicable regional equivalent) risk assessment, Business Associate Agreements with any infrastructure provider, and a documented breach-notification process.
- **Network isolation:** deployment inside a segmented, access-controlled network zone rather than a developer workstation or shared local network.
- **Model governance:** validation that the locally hosted LLM's outputs meet clinical-safety review requirements for the intended use case, plus a documented human-in-the-loop policy for any clinically consequential output.
- **Data retention & deletion:** defined retention periods and a verifiable deletion process for ingested documents and derived vectors.

13. Observability Architecture

Every query is logged with enough structured detail to diagnose retrieval or generation quality issues after the fact, without ever logging patient information (there is none, by design, in this phase — the same discipline carries forward regardless).

Category	Fields Logged
Query	Original query text, workflow_id, screen_id (if supplied)

Category	Fields Logged
Retrieval	Latency, raw Qdrant result count and scores, applied metadata filters
Reranking	Reranker scores, number of chunks kept vs. discarded
Generation	Ollama latency, model used, token usage (if reported by the model)
Trust	Grounded flag, confidence score/bucket, warnings, conflicts detected

Logs are structured (loguru, JSON-capable) so they can be shipped to a centralized log aggregator in a later phase without format changes.

14. Testing & Evaluation Strategy

14.1 Automated Test Coverage

Unit and integration tests exist for every layer of the pipeline:

- Document loading, chunking, and embedding generation
- Qdrant indexing, retrieval, and metadata filtering
- Reranking behavior
- Ollama generation and structured-output parsing
- Grounding scoring, citation correctness, and conflicting-document handling
- Prompt-injection detection

14.2 RAG Evaluation Framework

A lightweight evaluation harness runs a fixed set of synthetic questions (minimum 20) against the demo knowledge base (minimum 10 documents) and reports:

Metric	What it measures
Retrieval precision	Proportion of retrieved chunks that are actually relevant
Retrieval recall	Proportion of relevant chunks that were successfully retrieved
Context relevance	How well the assembled context matches the question's information need
Answer faithfulness	Whether the generated answer is fully supported by the supplied context
Citation correctness	Whether cited sources actually contain the claimed information
Hallucination rate	Frequency of unsupported claims in generated answers
Latency	End-to-end and per-stage response time

15. Non-Functional Requirements

Category	Requirement
Portability	No external AI service dependency; runs fully on local/on-prem infrastructure
Configurability	All models, thresholds, and endpoints changeable via environment variables only
Testability	Every layer independently unit-testable without the full stack running
Transparency	Every answer is traceable to specific cited source chunks
Safety	No confident answer without sufficient grounding evidence; conflicts surfaced, not resolved silently
Extensibility	Chunking, reranking, and hybrid (keyword + semantic) search can be upgraded without breaking the API contract
Observability	Every query stage produces structured, correlatable logs

16. Future State — Medical Agentic AI Integration

This RAG platform is designed from the outset to be consumed as a tool by a future Medical AI Agent, without requiring changes to its own architecture. The Agent brings planning and decision-making and orchestrates tools; the RAG platform continues to provide knowledge only.

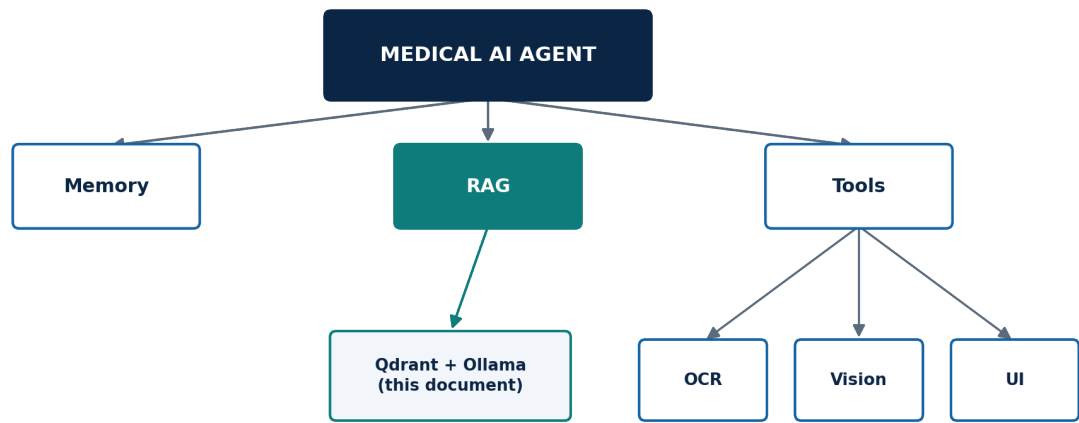


Figure 4 — The Agent orchestrates Memory, RAG, and Tools as peers; RAG remains the platform described in this document.

16.1 Agent-Facing Tool Surface

The future Agent is expected to call this platform through a small set of stable operations:

Operation	Purpose
rag.query()	Full grounded question-answering, screen/workflow-aware
rag.search()	Retrieval only, for Agent-side reasoning without generation cost
rag.get_screen_knowledge()	Convenience wrapper scoped to a specific screen_id
rag.get_workflow_instructions()	Convenience wrapper scoped to a specific workflow_id
rag.get_validation_rules()	Convenience wrapper for validation-rule document_type

16.2 Responsibility Boundaries

Layer	Provides	Explicitly does not provide
RAG (this platform)	Knowledge retrieval and grounded answers	Decisions, planning, or actions
Agent	Planning and decision-making	Direct knowledge storage or retrieval logic
Tools (OCR/Vision/UI)	Actions on the target application	Any reasoning about what action to take
Memory	State and prior context	Ground-truth medical/workflow knowledge

17. Risks, Assumptions & Open Items

17.1 Assumptions

- A capable Ollama model (7B+ parameter class or better) is available on the target host with sufficient RAM/VRAM for acceptable latency.

- Document volume during the prototype phase remains small enough (tens to low hundreds of documents) that a single-node Qdrant collection is sufficient without sharding.
- All knowledge-base content used during development and demonstration is synthetic or fully de-identified.

17.2 Risks

Risk	Mitigation
Local LLM output quality varies significantly by model choice	Model-agnostic design allows swapping models per environment; evaluation harness (Section 14) quantifies quality before any model is adopted
CPU-only embedding/reranking may be slow at larger corpus sizes	EMBEDDING_DEVICE and batch size are configurable; GPU acceleration can be enabled without code changes
Fixed-size chunking may split semantically related content	Semantic/parent-child chunking is architected as a drop-in upgrade path (Section 7.1)
Metadata quality depends on correct tagging at ingest time	Metadata extraction module centralizes and validates required fields via pydantic before indexing
Prompt-injection techniques evolve over time	Injection screening is isolated in its own guardrails module so detection logic can be updated independently of the rest of the pipeline
GPU VM cost/availability constraints on Azure	Sizing guidance (Section 10.2.4) starts conservative; scheduled start/stop or AKS GPU autoscaling (Section 10.2.5) can be adopted once usage patterns are known
Azure region GPU SKU availability varies	Confirm NCasT4_v3 (or chosen SKU) quota and availability in the target region before committing to an environment; fall back to an alternate region or GPU family if constrained

17.3 Open Items for Future Phases

- Hybrid (semantic + BM25 keyword) search — architected for, not yet enabled by default.
- Formal authentication/authorization implementation ahead of any non-synthetic data use.
- Centralized log aggregation and dashboarding for the observability layer.
- Load and latency benchmarking at realistic multi-user, multi-screen concurrency.

18. Appendix

18.1 Environment Configuration Reference

Variable	Default	Purpose
QDRANT_HOST / QDRANT_PORT	localhost / 6333	Qdrant connection
QDRANT_COLLECTION	medical_knowledge	Target collection name
OLLAMA_BASE_URL	http://localhost:11434	Ollama connection
OLLAMA_MODEL	(none hard-coded)	Locally installed model name, e.g. llama3.1, qwen2.5, mistral
EMBEDDING_MODEL	BAAI/bge-small-en-v1.5	Sentence Transformers model, shared by indexing and query
CHUNK_SIZE / CHUNK_OVERLAP	800 / 100	Fixed-size chunking parameters
TOP_K / RERANK_TOP_K	10 / 5	Retrieval funnel sizes
SCORE_THRESHOLD	0.3	Minimum similarity score accepted from Qdrant
RERANKER_MODEL / RERANKER_ENABLED	cross-encoder/ms-marco-MiniLM-L-6-v2 / true	Reranking configuration
GROUNDING_MIN_CONFIDENCE	0.5	Minimum confidence to return a generated answer rather than a fallback
AZURE_KEY_VAULT_URI	(production only)	Key Vault endpoint used to resolve secrets via managed identity in Azure
AZURE_LOG_ANALYTICS_WORKSPACE_ID	(production only)	Destination workspace for Azure Monitor log shipping

18.2 Reference Architecture Diagram Index

- Figure 1 — End-to-end ingestion and query pipelines (Section 6)
- Figure 2 — Data lifecycle through ingestion to Qdrant (Section 8.4)
- Figure 3 — Local development topology (Section 10.1)
- Figure 4 — Future Agent orchestration of Memory, RAG, and Tools (Section 16)
- Figure 5 — Layered functional view with cross-cutting concerns (Section 6.1)
- Figure 6 — Retrieval-to-generation funnel (Section 7.4)
- Figure 7 — Azure production deployment architecture (Section 10.2)

18.3 Glossary

Term	Definition
RAG	Retrieval-Augmented Generation — generating answers grounded in retrieved source content rather than model memory alone
Grounding	The property of an answer being verifiably supported by the retrieved context supplied to the LLM
Reranking	A second-stage relevance scoring pass, typically using a cross-encoder, applied after initial vector search
Payload metadata	Structured fields stored alongside each vector in Qdrant, used for filtering independent of semantic similarity
Agent	The future planning/decision-making system that will consume this RAG platform as a knowledge tool